

Name: Ankith Rangan

User-name: qbr56

Algorithm A: Genetic Algorithm

Algorithm B: Ant Colony Optimisation

Description of enhancement of Algorithm A:

First enhancement is using a hybrid ERX + OX crossover as opposed to the 'Single point crossover + repair' pseudocode shown in the lecture notes. Both Edge Recombination Crossover (ERX) and Order Crossover (OX) were implemented and experimentally compared. ERX here preserves high quality edge structures and generally produced shorter tours, while combining with OX introduced greater diversity.

Second Enhancement is adaptive crossover bias during stagnation. When no improvement was observed for a number of generations, probability of selecting ERX was reduced, increasing the influence of OX which was more destructive to edges, helping the population escape minima and reducing premature convergence.

Third enhancement is using a dynamic adaptive mutation rate. If improvement was detected, mutation was smoothly reduced to encourage exploitation. If stagnation, mutation rate increased within fixed bounds to encourage exploration, acting as a diversity mechanism.

Fourth enhancement is Selective Application of 2-Opt Local Search to the elite individuals of the population and probabilistically to newly generated offspring. Probabilistic approach chosen as it refines whilst maintaining diversity. A restricted neighbourhood (size 20) was used to reduce computation.

Fifth enhancement is Tournament Selection with Adaptive Pressure. During stagnation, tournament size is dynamically increased to increase selection pressure and speed up convergence when progress slowed.

Sixth enhancement is Inverse Mutation Operator. By reversing a subsequence of a tour instead of single swap, helped to provide more impactful variation than simple swap which could often break down tours.

Finally, the population size parameter and number of generations were tuned to balance computational cost with solution quality. Overall, my experimentation showed combining adaptive mechanisms with controlled local search produced consistently better tours than basic GA while remaining computationally feasible for big inputs.

Description of enhancement of Algorithm B:

First enhancement is introducing Stagnation Driven Adaptive Evaporation. Instead of using a fixed evaporation rate (which is standard in Ant System), evaporation parameter was dynamically adjusted based on stagnation. When no improvement observed for a number of iterations, ρ was increased within bounds. This strengthened evaporation, reduced pheromone dominance, discouraged aggressive convergence and encouraged exploration. When improvements occurred, ρ remained lower, preserving useful pheromone information. This helped prevent premature convergence while maintaining previously found good tours.

Second enhancement is Memetic ACO approach using selective 2-Opt local search. Within each iteration a restricted version of 2-Opt was applied only to the best tour of each iteration before updating pheromones, rather than to every ant. Different strategies were experimented and considered (applying 2-Opt to all ants or to several top ants), but this selective refinement provided good balance between computation cost and solution quality.

Third enhancement is using Max-Min pheromone bounding. Pheromone values restricted with upper and lower limits derived from best tour length. This prevented pheromone values becoming too large or disappearing too fast, reducing risk of stagnation and improving stability.

Fourth enhancement is introduction of a Dual Population Ant Role Framework. This is commented out on my code because it tended to run better on the 535 input city case, (ie better for longer tours), and without it the algorithm performed better for tours of size 50-100. The dual population allowed different combinations of alpha and beta for subsets of ants (exploiters, explorers), enabling differentiated search behaviour.

Finally, parameters eg num_ants, evaporation bounds and iteration limit were tuned according to problem size. Larger instances use capped ant populations to maintain computational strength.

DESCRIPTION OF ALGORITHM ONLY IF THE ALGORITHM IS NOT COVERED IN LECTURES

Description of *non-standard* Algorithm A:

Description of *non-standard* Algorithm B:

Type here, as above.